oVirt Setup

Architecture, Build & Setup Guide

Self-Hosted Engine on Bare-Metal Nodes

Document Version	1.0 — Initial Release
Target Environment	Lab / Self-Study
oVirt Version	4.5.x (Latest Stable)
OS Platform	Centos 8,9 / RHEL 8,9 Minimal
Network	192.168.5.0/24 — lab.local

1. Architecture Overview

This document defines the complete build and configuration guide for an oVirt lab environment using the Self-Hosted Engine (SHE) model. The oVirt Engine runs as a Virtual Machine (VM) on one of the physical hosts, managed by the cluster itself — eliminating the need for a dedicated physical management server.

1.1 Deployment Topology

> *Self-Hosted Engine: The oVirt Engine VM lives on oVirt1.lab.local and is managed and protected by the oVirt cluster itself. If Node 1 fails, the Engine VM can automatically migrate to Node 2 or Node 3.*

Key design decisions for this lab:

- Single management network (192.168.5.0/24) sufficient for lab use
- NFS shared storage serves both the data domain (VM disks) and the ISO domain
- Three compute hosts provide HA capability and live migration
- DNS is mandatory — all hostnames must resolve forward and reverse before installation

1.2 Infrastructure Component Summary

Component	Hostname	IP Address	Role	Notes
DNS Server	dns.lab.local	192.168.5.140	Name Resolution	BIND9
NFS Server	nfs.lab.local	192.168.5.141	Shared Storage	ISO + Data domains
oVirt Node 1	ovirt1.lab.local	192.168.5.142	Engine Host	Self-hosted engine primary
Engine VM	engine.lab.local	192.168.5.145	oVirt Engine	Self-hosted engine VM
oVirt Node 2	ovirt2.lab.local	192.168.5.143	Host	Additional compute node
oVirt Node 3	ovirt3.lab.local	192.168.5.144	Host	Additional compute node

2. Hardware & OS Requirements

2.1 Minimum Specifications

Component	vCPU/CPU	RAM	Disk (OS)	Disk (Data)
DNS Server	2 vCPU	2 GB	20 GB	—
NFS Server	2 vCPU	4 GB	20 GB	200 GB+
oVirt Node 1–3	8+ CPU cores	32 GB+	100 GB	—
Engine VM	4 vCPU (auto)	16 GB (auto)	60 GB (auto)	—

> *Hosts must support hardware virtualization (Intel VT-x or AMD-V). Verify with: grep -E 'vmx|svm' /proc/cpuinfo*

2.2 Operating System

All nodes and servers in this lab run Centos 8 (or RHEL 8). The oVirt 4.5 repositories are compatible with EL8-based distributions.

- AlmaLinux 8 / Rocky Linux 8 / RHEL 8 — supported
- Kernel: 4.18+ (default EL8 kernel is sufficient)
- SELinux: Enforcing (do NOT disable — oVirt is SELinux-aware)
- Firewalld: Active (oVirt manages its own zones)

3. DNS Server Setup — dns.lab.local (192.168.5.140)

DNS is a hard prerequisite for oVirt. Both forward (A) and reverse (PTR) records must be in place and resolving correctly on all nodes before any oVirt component is installed.

3.1 DNS Record Requirements

Hostname	Record Type	Value	Purpose
dns.lab.local	A	192.168.5.140	DNS Server
nfs.lab.local	A	192.168.5.141	NFS Storage
ovirt1.lab.local	A	192.168.5.142	Node 1 / Engine Host
engine.lab.local	A	192.168.5.145	oVirt Engine VM
ovirt2.lab.local	A	192.168.5.143	Node 2
ovirt3.lab.local	A	192.168.5.144	Node 3
140.5.168.192.in-addr.arpa	PTR	dns.lab.local	Reverse DNS
141.5.168.192.in-addr.arpa	PTR	nfs.lab.local	Reverse DNS
142.5.168.192.in-addr.arpa	PTR	ovirt1.lab.local	Reverse DNS
145.5.168.192.in-addr.arpa	PTR	engine.lab.local	Reverse DNS
143.5.168.192.in-addr.arpa	PTR	ovirt2.lab.local	Reverse DNS
144.5.168.192.in-addr.arpa	PTR	ovirt3.lab.local	Reverse DNS

3.2 Installation & Configuration

1. Install BIND on the DNS server:

```
dnf install -y bind bind-utils
systemctl enable --now named
```

2. Configure /etc/named.conf — add the zone definitions:

```
zone "lab.local" IN {
    type master;
    file "/var/named/lab.local.zone";
    allow-update { none; };
};

zone "5.168.192.in-addr.arpa" IN {
    type master;
    file "/var/named/5.168.192.arpa";
    allow-update { none; };
};
```

3. Create forward zone file /var/named/lab.local.zone:

```
$TTL 86400
@   IN SOA dns.lab.local. admin.lab.local. (
              2024010101 ; Serial
              3600       ; Refresh
              1800       ; Retry
              604800     ; Expire
              86400 )    ; Minimum TTL

    IN NS   dns.lab.local.

dns     IN A  192.168.5.140
nfs     IN A  192.168.5.141
ovirt1  IN A  192.168.5.142
ovirt2  IN A  192.168.5.143
ovirt3  IN A  192.168.5.144
engine  IN A  192.168.5.145
```

4. Create reverse zone file /var/named/5.168.192.arpa:

```
$TTL 86400
@   IN SOA dns.lab.local. admin.lab.local. (
              2024010101 3600 1800 604800 86400 )
    IN NS   dns.lab.local.

140 IN PTR dns.lab.local.
141 IN PTR nfs.lab.local.
142 IN PTR ovirt1.lab.local.
143 IN PTR ovirt2.lab.local.
144 IN PTR ovirt3.lab.local.
145 IN PTR engine.lab.local.
```

5. Open firewall and restart named:

```
firewall-cmd --permanent --add-service=dns
firewall-cmd --reload
systemctl restart named
```

3.3 DNS Verification (Run on ALL Nodes)

Configure /etc/resolv.conf on every host to point to 192.168.5.140, then verify:

```
# Set DNS resolver on all nodes
echo 'nameserver 192.168.5.140' > /etc/resolv.conf

# Forward lookup tests
nslookup engine.lab.local
nslookup ovirt1.lab.local

# Reverse lookup tests
nslookup 192.168.5.145
nslookup 192.168.5.142

# All six hosts must resolve forward AND reverse
```

4. NFS Server Setup — nfs.lab.local (192.168.5.141)

NFS provides a shared storage backend for oVirt. Three separate exports are required: one for the self-hosted engine VM storage, one for VM data disks, and one for ISO images.

4.1 NFS Export Configuration

Export Path	Clients	Purpose
/exports/ovirt-data	192.168.5.0/24(rw,sync,no_root_squash)	VM Data Storage Domain
/exports/ovirt-iso	192.168.5.0/24(rw,sync,no_root_squash)	ISO Storage Domain
/exports/ovirt-hosted-engine	192.168.5.0/24(rw,sync,no_root_squash)	Self-Hosted Engine Storage

4.2 Installation & Configuration

6. Install NFS server packages:

```
dnf install -y nfs-utils
systemctl enable --now nfs-server rpcbind
```

7. Create export directories and set up correct ownership:

```
mkdir -p /exports/ovirt-data
mkdir -p /exports/ovirt-iso
mkdir -p /exports/ovirt-hosted-engine

# oVirt requires vdsm:kvm ownership (UID 36, GID 36)
chown -R 36:36 /exports/ovirt-data
chown -R 36:36 /exports/ovirt-iso
chown -R 36:36 /exports/ovirt-hosted-engine
chmod 0755 /exports/ovirt-data
chmod 0755 /exports/ovirt-iso
chmod 0755 /exports/ovirt-hosted-engine
```

8. Edit /etc/exports:

```
/exports/ovirt-data          192.168.5.0/24(rw,sync,no_root_squash,no_subtree_check)
/exports/ovirt-iso           192.168.5.0/24(rw,sync,no_root_squash,no_subtree_check)
/exports/ovirt-hosted-engine 192.168.5.0/24(rw,sync,no_root_squash,no_subtree_check)
```

9. Apply exports and configure firewall:

```
exportfs -rav
```

```
firewall-cmd --permanent --add-service=nfs
firewall-cmd --permanent --add-service=rpc-bind
firewall-cmd --permanent --add-service=mountd
firewall-cmd --reload
```

4.3 NFS Verification

```
# From any oVirt node, verify NFS exports are visible
showmount -e nfs.lab.local

# Expected output:
# Export list for nfs.lab.local:
# /exports/ovirt-hosted-engine  192.168.5.0/24
# /exports/ovirt-iso            192.168.5.0/24
# /exports/ovirt-data           192.168.5.0/24
```

5. oVirt Host Preparation (All Nodes: ovirt1, ovirt2, ovirt3)

Perform the following steps on ALL THREE hosts: ovirt1.lab.local (192.168.5.142), ovirt2.lab.local (192.168.5.143), and ovirt3.lab.local (192.168.5.144).

5.1 System Prerequisites

10. Set correct hostname (example for Node 1):

```
hostnamectl set-hostname ovirt1.lab.local
# Repeat with appropriate names for ovirt2 and ovirt3
```

11. Point DNS resolver to 192.168.5.140:

```
nmcli con mod <interface> ipv4.dns 192.168.5.140
nmcli con up <interface>

# Verify hostname resolution
hostname -f   # Must return FQDN
nslookup $(hostname -f)   # Must resolve
```

12. Disable chronyd or configure NTP (all hosts must be time-synced):

```
dnf install -y chrony
systemctl enable --now chronyd
chronyc sources
```

13. Enable required kernel modules and disable transparent hugepages:

```
# Verify virtualization support
grep -E 'vmx|svm' /proc/cpuinfo | head -2

# Disable THP for better VM performance
echo never > /sys/kernel/mm/transparent_hugepage/enabled
echo never > /sys/kernel/mm/transparent_hugepage/defrag
```

5.2 Add oVirt Repository

```
dnf install -y https://resources.ovirt.org/pub/yum-repo/ovirt-release45.rpm
dnf module enable -y javapackages-tools
dnf module enable -y pki-deps
dnf distro-sync -y
```

5.3 Install oVirt Host Packages

```
dnf install -y ovirt-host

# The ovirt-host meta-package installs:
# - VDSM (Virtual Desktop and Server Manager)
# - libvirt, QEMU/KVM
# - oVirt network and storage agents
# - cockpit-ovirt-dashboard
```

> *Do NOT start or configure VDSM manually. The oVirt Engine will bootstrap and manage VDSM on each host during the hosted-engine deployment and Add Host wizard.*

5.4 Network & Firewall

```
# oVirt manages its own firewall rules via firewalld
# Ensure firewalld is active
systemctl enable --now firewalld

# Verify the ovirtmgmt zone exists (created by VDSM)
firewall-cmd --get-zones | grep ovirt
```

6. oVirt Engine Deployment — Self-Hosted Engine

The Self-Hosted Engine (SHE) is deployed from ovirt1.lab.local. The hosted-engine setup wizard will create the Engine VM automatically, store it on the NFS hosted-engine export, and register ovirt1 as the first host in the cluster.

6.1 Install Hosted Engine on Node 1

```
# Run on ovirt1.lab.local ONLY
dnf install -y ovirt-hosted-engine-setup
```

6.2 Run the Hosted Engine Setup Wizard

```
hosted-engine --deploy
```

The interactive wizard will prompt for the following values. Use the table below as your answer reference:

Wizard Prompt	Value to Enter
Storage type	nfs
NFS server	nfs.lab.local
NFS export path (engine storage)	/exports/ovirt-hosted-engine
NFS version	4.1 (or auto)
Engine VM FQDN	engine.lab.local
Engine VM IP address	192.168.5.145
Engine admin password	<choose a strong password>
CPU type	host-passthrough (recommended for lab)
Memory for Engine VM	16384 (MB) — minimum 4096
Disk size for Engine VM	60 (GB) — minimum 60
Bridge interface	eth0 / ens3 (your main interface name)
Management network bridge	ovirtmgmt

> *The wizard takes 20–40 minutes. It will deploy the Engine VM, install oVirt Engine inside it, and configure ovirt1 as the first host. Watch the progress with: tail -f /var/log/ovirt-hosted-engine-setup/ovirt-hosted-engine-setup-*.log*

6.3 Post-Deployment Verification

```
# Check engine VM status
```

```
hosted-engine --vm-status

# Check overall hosted engine health
hosted-engine --check-liveliness

# Access the Engine Web UI
# https://engine.lab.local/ovirt-engine
# Username: admin
# Domain: internal
# Password: <password set during wizard>
```

7. Adding Hosts to the Cluster (ovirt2 & ovirt3)

After the Engine is running and accessible, add ovirt2 and ovirt3 through the Web Administration Portal. The Engine will SSH into each host, install and configure VDSM, and bring the host into the cluster.

7.1 Add Host via Web UI

14. Open https://engine.lab.local/ovirt-engine and log in as admin.
15. Navigate to: Compute → Hosts → New.
16. Fill in the host details:

Field	ovirt2	ovirt3
Name	ovirt2	ovirt3
Hostname/IP	ovirt2.lab.local	ovirt3.lab.local
SSH Port	22	22
Authentication	Password or SSH Key	Password or SSH Key
Cluster	Default (or your cluster)	Default (or your cluster)

17. Click OK. The Engine will install VDSM and configure the host. Monitor status in Compute → Hosts until state shows 'Up'.

7.2 Host Status Verification

```
# On each node, verify VDSM is running after engine enrollment
systemctl status vdsmd


# Check host is Up in engine
# Compute → Hosts → ovirt2 → General → Status = Up
```

8. Configuring Storage Domains

Storage domains must be added from the Engine Web UI after all hosts are Up. The hosted-engine storage is already configured. Add the data and ISO domains manually.

8.1 Add NFS Data Storage Domain

18. Navigate to: Storage → Domains → New Domain.
19. Configure the Data domain:
- Name: ovirt-data
- Domain Function: Data
- Storage Type: NFS
- NFS Server Address: nfs.lab.local
- NFS Path: /exports/ovirt-data
- NFS Version: 4.1
20. Click OK and wait for status: Active.

8.2 Add NFS ISO Storage Domain

21. Navigate to: Storage → Domains → New Domain.
22. Configure the ISO domain:
- Name: ovirt-iso
- Domain Function: ISO
- Storage Type: NFS
- NFS Server Address: nfs.lab.local
- NFS Path: /exports/ovirt-iso
23. Click OK and wait for status: Active.

8.3 Upload ISO Images

```
# Upload ISOs from any host using ovirt-iso-uploader (legacy)
# Or use the Web UI: Storage → Domains → ovirt-iso → Images → Upload

# Alternative: Copy ISO directly to NFS export (quick lab method)
cp /path/to/rhel8.iso /exports/ovirt-iso/
# Then refresh the ISO domain in Web UI
```

9. Network Port Reference

Service	Port	Protocol	Description
HTTPS	443	TCP	oVirt Engine Web UI & REST API
HTTP	80	TCP	Redirect to HTTPS
SPICE	5900-6923	TCP	VM Console (SPICE)
VNC	5634-6166	TCP	VM Console (VNC)
VDSM	54321	TCP	Host ↔ Engine communication
libvirt	16514	TCP	Live migration
Migration	49152-49216	TCP	VM live migration
NFS	2049	TCP/UDP	Storage domain access
DNS	53	TCP/UDP	Name resolution
SSH	22	TCP	Host management

> *For a lab environment with a single flat network, no additional firewall rules are typically needed beyond the defaults that oVirt manages through firewalld. These ports are listed for reference when moving to production or when troubleshooting connectivity.*

10. Full Deployment Verification Checklist

#	Verification Step	Expected Result
1	nslookup engine.lab.local from all hosts	Returns 192.168.5.145
2	nslookup 192.168.5.145 from all hosts	Returns engine.lab.local
3	showmount -e nfs.lab.local from all hosts	Shows all 3 export paths
4	hosted-engine --vm-status on ovirt1	Engine VM = Up
5	hosted-engine --check-liveliness	Engine is alive
6	Web UI accessible: https://engine.lab.local/ovirt-engine	Login page loads
7	Compute → Hosts shows all 3 nodes	Status = Up for all
8	Storage → Domains shows data + ISO domains	Status = Active
9	Create a test VM with disk on ovirt-data	VM starts successfully
10	Live migrate test VM between hosts	Migration completes

11. Common Issues & Troubleshooting

11.1 DNS Failures

Symptom: 'Failed to resolve host' during hosted-engine deploy

```
# Check /etc/resolv.conf points to 192.168.5.140
cat /etc/resolv.conf

# Check BIND is running on DNS server
systemctl status named

# Test from problem host
dig engine.lab.local @192.168.5.140
```

11.2 NFS Mount Failures

Symptom: Storage domain stuck in 'Inactive' or 'Locked'

```
# Verify NFS exports from engine or host
showmount -e nfs.lab.local

# Check ownership of NFS directories on NFS server
```

```
ls -la /exports/  # Must show UID 36 (vdsm)

# Manually test mount
mount -t nfs4 nfs.lab.local:/exports/ovirt-data /mnt
ls /mnt  # Should succeed
umount /mnt
```

11.3 Engine VM Not Starting

```
# Check hosted engine agent logs
journalctl -u ovirt-ha-agent -f
journalctl -u ovirt-ha-broker -f

# Force engine VM start (if HA agent is stuck)
hosted-engine --vm-start

# Full hosted engine logs
ls /var/log/ovirt-hosted-engine-setup/
```

11.4 Host Stuck in 'Installing' or 'Non-Operational'

```
# On the problem host — check VDSM
systemctl status vdsmd
journalctl -u vdsmd -n 100

# Verify time sync (clock skew causes auth failures)
chronyc tracking

# Check SELinux is not blocking VDSM
ausearch -m avc -ts recent | grep vdsm
```

Appendix: Quick Reference

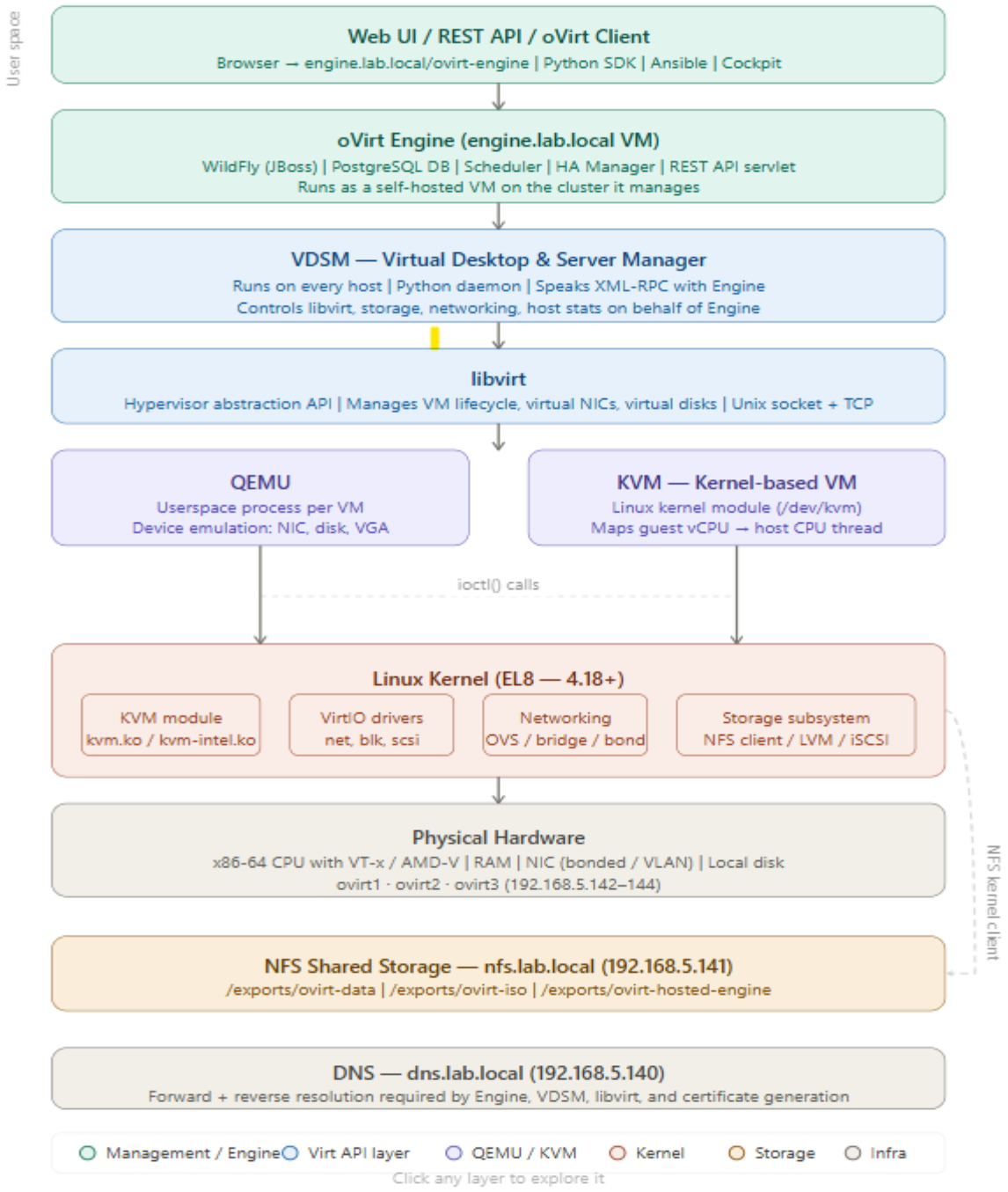
Key Service Locations

Item	Location / Command
Engine Web UI	https://engine.lab.local/ovirt-engine

Item	Location / Command
Engine logs	/var/log/ovirt-engine/engine.log
VDSM logs (hosts)	/var/log/vdsm/vdsm.log
Hosted engine logs	/var/log/ovirt-hosted-engine-setup/
HA agent logs	journalctl -u ovirt-ha-agent
VDSM service	systemctl status vdsmd
Hosted engine status	hosted-engine --vm-status
Engine health check	hosted-engine --check-liveliness
Engine DB (postgres)	engine-db-query (on engine VM)
REST API	https://engine.lab.local/ovirt-engine/api

12. Software Architecture — Kernel to User Space

oVirt does not implement its own hypervisor. It orchestrates a layered stack of Linux-native virtualization components. The table and descriptions below map every software layer from physical hardware up to the Web UI. Reading bottom-up mirrors how control actually flows: hardware capabilities are exposed by the kernel, abstracted by KVM and QEMU, managed by libvirt, brokered by VDSM, coordinated by the Engine, and surfaced to operators through the Web UI or REST API.



12.1 Layer Reference Table

Layer	Layer Name	Key Components	Host / Location	Role and Key Technical Facts
7	**Web UI / Client**	Browser, Ansible, Python SDK, Cockpit	Admin workstation	Communicates via HTTPS REST API on port 443. Cockpit-ovirt dashboard runs on each host port 9090.
6	**oVirt Engine**	WildFly (JBoss), PostgreSQL, HA Manager, REST servlet	engine.lab.local (192.168.5.145)	Self-hosted VM. Stores all cluster state in PostgreSQL. Schedules VMs, enforces HA, issues commands to VDSM agents via XML-RPC on port 54321.
5	**VDSM**	vdsmd (Python daemon), ovirt-ha-agent, ovirt-ha-broker	ovirt1 / ovirt2 / ovirt3	Runs on every host. Acts as the Engine's local agent. Translates high-level commands into libvirt API calls and kernel operations. Also manages storage and networking config on the host.
4	**libvirt**	libvirtd, virsh, libvirt XML domain API	ovirt1 / ovirt2 / ovirt3	Hypervisor abstraction layer. Manages VM lifecycle (define, start, stop, migrate) via a stable XML API. Speaks to QEMU via Unix socket. Enables live migration between hosts via TCP port 16514.
3	**QEMU + KVM**	qemu-kvm (process per VM), kvm.ko, kvm-intel.ko / kvm-amd.ko	ovirt1 / ovirt2 / ovirt3	QEMU runs as a userspace process per VM and emulates all virtual devices (NIC, disk, VGA). KVM is a kernel module that maps guest vCPU to host CPU threads using VT-x/AMD-V hardware extensions. QEMU calls KVM via ioctl() on /dev/kvm for CPU and memory virtualization.
2	**Linux Kernel (EL8)**	kvm.ko, virtio drivers, OVS/bridge/bond, NFS client, LVM	All nodes	The kernel IS the hypervisor platform. VirtIO drivers (virtio-net, virtio-blk) give guest VMs near-native I/O performance. The NFS kernel client mounts shared storage. Firewalld / nftables enforce port rules. SELinux enforces mandatory access control on VDSM and QEMU processes.
1	**Physical Hardware**	x86-64 CPU, RAM, NIC (bonded/VLAN), local disk	ovirt1 / ovirt2 / ovirt3	CPUs must expose VT-x (Intel) or AMD-V extensions. IOMMU (VT-d / AMD-Vi) enables PCI passthrough. NIC bonding provides redundancy for the ovirtmgmt management network. Local disk holds the host OS; VM data lives on NFS shared storage.

S	**NFS Shared Storage**	nfs-server, exportfs	nfs.lab.local (192.168.5.141)	Three exports: /ovirt-hosted-engine (Engine VM disk), /ovirt-data (VM data domain), /ovirt-iso (ISO images). Ownership must be UID/GID 36 (vdsm:kvm). Accessed by all hosts via NFS v4.1 kernel client.
D	**DNS**	named (BIND9)	dns.lab.local (192.168.5.140)	Forward (A) and reverse (PTR) records required for all 6 hosts. Used by Engine certificate generation, VDSM registration, and libvirt migration. Must resolve before any oVirt component is installed.

12.2 Layer 1 — Physical Hardware

The foundation of the stack. All three oVirt nodes (ovirt1, ovirt2, ovirt3) must have x86-64 CPUs with hardware virtualization extensions enabled in the BIOS. Intel CPUs need VT-x for CPU virtualization and VT-d for IOMMU/PCI passthrough. AMD CPUs need AMD-V and AMD-Vi. Without these extensions, the KVM kernel module will fail to load and no VMs can run.

> *Verify with: grep -E 'vmx|svm' /proc/cpuinfo — vmx = Intel VT-x, svm = AMD-V. No output means virtualization is disabled in BIOS.*

12.3 Layer 2 — Linux Kernel (EL8 4.18+)

AlmaLinux 8 / RHEL 8 ships with kernel 4.18 which includes KVM as a built-in module. The kernel is what makes Linux a Type-1 hypervisor — no separate hypervisor OS is needed. Key subsystems in use: KVM module (/dev/kvm device), VirtIO drivers for near-native guest I/O, the NFS kernel client for shared storage mounts, Open vSwitch or Linux bridge for the ovirtmgmt network, and SELinux for mandatory access control over QEMU and VDSM processes.

```
# Verify KVM modules are loaded on every host
lsmod | grep kvm
# Expected output:
# kvm_intel   (or kvm_amd)
# kvm

# Check /dev/kvm exists
ls -la /dev/kvm
```

12.4 Layer 3 — QEMU + KVM (The Actual Hypervisor)

QEMU and KVM always work as a pair. KVM (kernel module) handles privileged operations: mapping guest virtual CPUs to host CPU threads, managing guest physical memory pages, and intercepting sensitive CPU instructions from the guest OS. QEMU (userspace process, one per VM) handles all device emulation: the virtual NIC, virtual SCSI/SATA controller, video adapter, USB, and serial ports. QEMU calls KVM via ioctl() on /dev/kvm to offload CPU and memory virtualization to hardware, making VMs run at near-native speed.

> *Each running VM = one qemu-kvm process on the host. You can see them with: ps aux | grep qemu*

```
# List all running VM processes on a host
ps aux | grep qemu-kvm

# Check KVM acceleration is active for a VM
virsh dominfo <vm-name> | grep -i cpu
```

12.5 Layer 4 — libvirt

libvirt is the hypervisor abstraction API. It sits between VDSM and QEMU/KVM, providing a stable XML-based domain API for creating, starting, stopping, and migrating VMs. libvirt manages: VM definitions (XML domain files in /etc/libvirt/qemu/), virtual network bridges, virtual disk attachment, and live migration negotiation between hosts. VDSM calls libvirt via a Unix socket (/var/run/libvirt/libvirt-sock). libvirt then drives QEMU directly.

```
# List all VMs known to libvirt on a host
virsh list --all

# Inspect a VM's XML definition
virsh dumpxml <vm-name>

# Live migrate a VM from ovirt1 to ovirt2
virsh migrate --live <vm> qemu+ssh://ovirt2.lab.local/system
```

12.6 Layer 5 — VDSM

VDSM (Virtual Desktop and Server Manager) is oVirt's host agent. It runs as a Python daemon (vdsmd) on every host and is the only process the Engine communicates with directly. VDSM responsibilities: receiving commands from Engine via XML-RPC on port 54321, translating those commands into libvirt API calls, managing storage domain connections (NFS mounts), configuring host networking (bonds, VLANs, bridges), reporting host health statistics to the Engine, and managing the self-hosted Engine HA state via ovirt-ha-agent and ovirt-ha-broker.

```
# Check VDSM status on any host
systemctl status vdsmd

# Tail VDSM logs (most useful for troubleshooting)
tail -f /var/log/vdsm/vdsm.log

# Check HA agent (hosted engine health)
systemctl status ovirt-ha-agent
hosted-engine --vm-status
```

12.7 Layer 6 — oVirt Engine

The Engine is the central brain of the cluster. It runs as a Java web application inside WildFly (JBoss application server) on the Engine VM (engine.lab.local, 192.168.5.145). It stores all cluster state — hosts, VMs, storage

domains, networks, users — in a PostgreSQL database. The Engine's scheduler decides which host runs each VM based on policy (CPU load, memory, affinity rules). The Engine exposes a REST API consumed by the Web UI, Python SDK, Ansible ovirt.ovirt collection, and Terraform.

> *The Engine VM is self-hosted: it runs as a VM on the cluster it manages. If ovirt1 fails, the HA agent migrates the Engine VM to ovirt2 or ovirt3 automatically.*

```
# Engine service status (run inside engine VM)
systemctl status ovirt-engine


# Engine logs
tail -f /var/log/ovirt-engine/engine.log


# REST API — list all VMs
curl -u admin@internal https://engine.lab.local/ovirt-engine/api/vms
```

12.8 Layer 7 — Web UI / REST API / Clients

The top layer is everything that talks to the Engine's REST API. The oVirt Administration Portal is a GWT web application served by the Engine at https://engine.lab.local/ovirt-engine. Operators use it for day-to-day VM management. For automation: the Python SDK (ovirt-engine-sdk-python) wraps the REST API, Ansible uses the ovirt.ovirt certified collection, and Terraform has an oVirt provider. The Cockpit oVirt dashboard (port 9090 on each host) provides host-level management without needing the full Engine.

> *For Red Hat Certified Architect exam scenarios: oVirt's REST API is the integration point for all automation. Know the API endpoint structure: /ovirt-engine/api/vms, /api/hosts, /api/storagedomains.*

12.9 Call Flow: What Happens When You Click “Start VM”

Tracing a single operation through all 7 layers reveals how the components interact in real time:

```
1.  Browser (Layer 7) → HTTPS POST to /ovirt-engine/api/vms/{id}/start
2.  Engine REST servlet (Layer 6) validates auth, checks scheduler policy
3.  Engine selects target host (e.g. ovirt2) based on CPU/memory availability
4.  Engine sends XML-RPC 'create' command to VDSM on ovirt2 (port 54321)
5.  VDSM (Layer 5) on ovirt2 receives command, builds libvirt XML domain definition
6.  VDSM calls libvirt API: virDomainCreate(xml_definition)
7.  libvirt (Layer 4) on ovirt2 parses XML, launches qemu-kvm process
8.  QEMU (Layer 3) initialises virtual devices, opens ioctl() to /dev/kvm
9.  KVM (Layer 3) allocates guest memory pages, creates vCPU threads
10. Linux kernel (Layer 2) schedules vCPU threads on physical CPU cores
11. Hardware VT-x (Layer 1) enforces guest/host privilege separation
12. VDSM polls VM state, reports 'Up' back to Engine
13. Engine updates PostgreSQL, pushes status to Web UI via long-poll
14. Browser displays VM status as Running
```